

NOCTILUZ

CANAL DIGITAL DE VENTA INSTRUMENTADO PARA UNA TIENDA DE MODA LUMINOSA URBANA

Memoria de la Tarea 1 · Soluciones Informáticas para la Empresa · Grado en Ingeniería Informática · UCAM · Curso 2026-2027

Integrantes del grupo

Luis Galindo Buendía · Roberto Ciller Miñana
Javier Zamora Lopez · Jun-Jie Jiang Yao

URL pública: noctiluz.free.je (*despliegue provisional*)

Repositorio: github.com/luisgabu0-gif/noctiluz-ucam

Prototipo académico sin actividad comercial real. Todos los datos son ficticios.

1 Caso de empresa y arquitectura

1.1 Caso de empresa

NOCTILUZ es una marca ficticia de moda urbana con iluminación integrada (camisetas, chaquetas, mochilas y gorras con fibra óptica y módulo LED extraíble). El catálogo se organiza por **tipo de prenda**, **color de luz** (cian, rosa, ámbar, violeta, verde lima y blanco frío) y **modo de iluminación** (fijo, parpadeo y degradado): 16 productos, cada uno con una variante (SKU) en un color de luz y un único modo de iluminación. Como cada producto solo admite el modo que tiene asignado, la combinación producto-color-modo-talla es una regla de negocio que valida el servidor, no un simple filtro visual.

El canal cubre el recorrido completo (catálogo, carrito, *checkout*, pago simulado, pedido y seguimiento) y genera los datos y eventos que consumirán otros sistemas en la Tarea 2. Condiciones simuladas: IVA del 21 % incluido y desglosado, envío estándar de 4,95 € (gratis desde 120 €) o exprés de 9,95 €, cupones con importe mínimo, máximo de 9 unidades por línea y stock por variante.

1.2 Arquitectura

Arquitectura cliente-servidor en capas para un alojamiento compartido con Apache, PHP y MySQL; desplegada de forma provisional en InfinityFree y migrable a DonDominio sin cambiar el código.

- **Interfaz** (`index.html`, `admin.html`, `assets/js`): aplicación de una página con HTML, CSS y JavaScript nativo en módulos. Solo presenta datos y recoge acciones.
- **Servicios** (`api/`): un *endpoint* PHP por recurso, con JSON y errores de negocio como respuestas 422 con detalle por campo.
- **Lógica de negocio** (`app/`): clases `ReglasNegocio`, `Pedidos`, `Pagos`, `Incidencias`, `Catalogo` y `Eventos`. Precios, descuentos, impuestos y envío se calculan siempre desde la base de datos, nunca desde lo que envía el navegador.
- **Persistencia** (`app/Db.php`, MySQL): PDO con sentencias preparadas. Crear un pedido (cliente, pedido, líneas, stock, historial y evento) es una única transacción.

El carrito vive en el navegador (aún no es un compromiso de compra); al confirmar, el servidor lo revalida por completo (existencia, modo, talla, cantidad y stock).

1.3 Decisiones tecnológicas y alternativas

Aspecto	Decisión	Alternativas	Justificación
Servidor	PHP 8 sin <i>framework</i>	Node + Express; Laravel; Flask	El hosting básico ejecuta PHP nativo; Node o Python exigirían un VPS. Un <i>framework</i> añade dependencias que no compensan para 12 <i>endpoints</i> .
Plataforma	Desarrollo propio	WooCommerce, PrestaShop	Resuelven el carrito pero ocultan el modelo de datos y la instrumentación, que son el objeto de la tarea.
Base de datos	MySQL (SQLite en local)	Ficheros JSON; Firebase	Relacional con claves foráneas, restricciones y transacciones, incluida en el hosting. SQLite con el mismo SQL permite ejecutar el proyecto sin servidor de BD.
Interfaz	HTML, CSS y JS nativo	React, Vue	Se partía de un HTML existente; un <i>framework</i> obligaría a compilar sin aportar funcionalidad necesaria.
Pago	Pasarela simulada propia	Stripe o Redsys en pruebas	Evita cuentas y claves de terceros (el enunciado prohíbe credenciales). Solo guarda los 4 últimos dígitos.
Eventos	Tabla en BD + API con cursor	Log; Google Analytics; cola de mensajes	Se guarda en la misma transacción que el dato de negocio y se consulta con SQL. La analítica externa no ve eventos de servidor y cede datos; una cola exige infraestructura que no hay en hosting compartido.
Clientes	Email + código de pedido	Registro con contraseña	Menos datos personales y flujo más simple; suficiente para un prototipo.
Alojamiento	InfinityFree (provisional)	DonDominio; Vercel	Vercel no ejecuta PHP nativo ni ofrece MySQL. InfinityFree ofrece lo mismo que DonDominio (Apache, PHP 8, MySQL, <code>.htaccess</code>) sin coste; migrar solo exige reimportar y resubir.

2 Modelo de datos

El diagrama entidad-relación en notación de Chen (Figura 1) tiene una única relación N:M, PRODUCTO-MODO (modos admitidos por modelo), que en el esquema relacional pasa a la tabla `producto_modos`. La compra de una variante con modo y talla es la entidad LÍNEA DE PEDIDO, que guarda cantidad y precio del momento. El esquema relacional completo son 15 tablas (Figura 2).

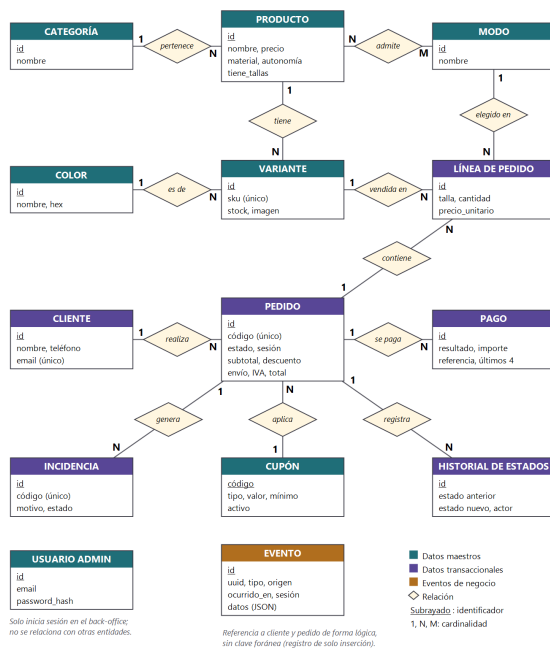


Figura 1: Diagrama entidad-relación (notación de Chen).

Datos maestros. categorías, colores y modos (criterios del catálogo); productos; producto_modos (N:M); variantes (SKU con stock e imagen); cupones; usuarios_admin.

Datos transaccionales. clientes (email único); pedidos (código único NCZ-AAMDD-XXXXXX, estado, desglose de importes y copia de la dirección); lineas_pedido (descripción y precio del momento, para que un cambio posterior no altere pedidos pasados); pagos (cada intento); historial_estados (quién, cuándo y por qué); incidencias.

Eventos. eventos no tiene claves foráneas a propósito: es un registro de solo inserción que se conserva aunque se archiven o borren los datos transaccionales.

Integridad. Claves foráneas; únicos (SKU, email, código de pedido, referencia de pago, *uuid*); CHECK (`stock >= 0`); descuento de stock condicionado (`UPDATE ... WHERE stock >= cantidad`) contra ventas simultáneas; importes en céntimos enteros.

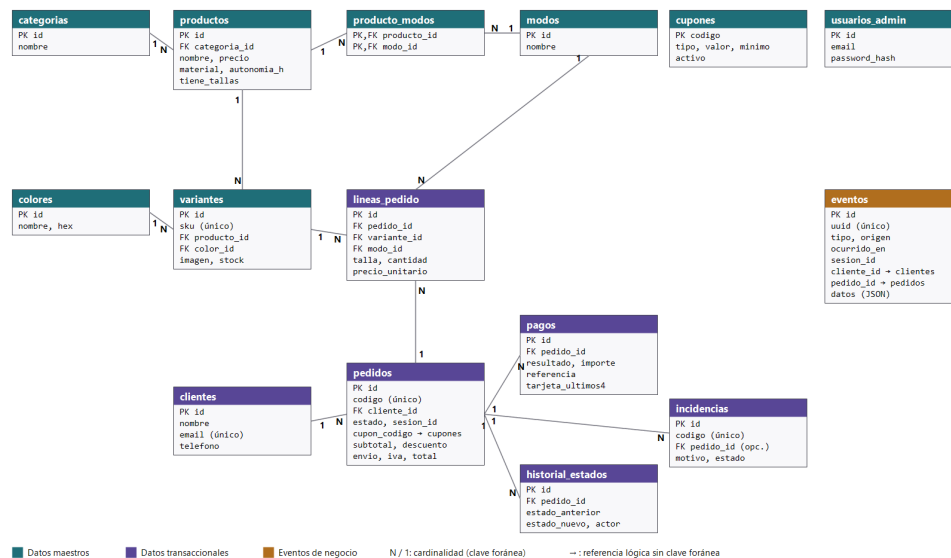


Figura 2: Esquema relacional (15 tablas), generado a partir de `database/esquema_mysql.sql`.

El ciclo de vida del pedido es una máquina de estados (Tabla 1).

Estado	Transiciones permitidas
creado	pagado (solo la pasarela simulada), cancelado
pagado	en_preparacion, enviado, entregado, incidencia, cancelado
en_preparacion	enviado, entregado, incidencia, cancelado
enviado	entregado, incidencia
entregado	incidencia
incidencia	en_preparacion, enviado, entregado, cancelado
cancelado	ninguna (estado final; devuelve el stock)

Tabla 1: Máquina de estados del pedido.

3 Eventos registrados

Evento	Origen	Momento y datos principales
product.viewed	Navegador	Apertura de una ficha. SKU y precio.
cart.item_added	Navegador	Línea añadida al carrito. SKU, modo, talla, cantidad y precio.
cart.item_removed	Navegador	Línea eliminada del carrito. SKU y cantidad.
checkout.started	Navegador	Entrada en el <i>checkout</i> . Nº de líneas, unidades y subtotal.
order.created	Servidor	Creación del pedido, en su misma transacción. Código, total, líneas, envío y cupón.
payment.simulated	Servidor	Cada intento de pago. Resultado, importe y referencia.
order.status_changed	Servidor	Cada transición de estado. Estado anterior y nuevo, actor y motivo.
support.requested	Servidor	Solicitud de soporte o incidencia. Código, motivo y pedido asociado.

Tabla 2: Catálogo de eventos instrumentados.

Cómo se generan. Los eventos de interfaz los envía `eventos.js` a `POST api/eventos.php`, que admite solo una lista cerrada de tipos, valida el identificador de sesión y limita tamaño y formato. Los eventos con consecuencias de negocio los genera únicamente el servidor dentro de la misma transacción que el cambio de datos: no puede existir un pedido sin su `order.created` ni al revés.

Dónde se almacenan. En la tabla `eventos`, con un sobre común: `uuid`, `tipo`, `origen`, `ocurrido_en`, `sesion_id`, `cliente_id`, `pedido_id` y `datos` (JSON). La sesión es un UUID aleatorio generado en el navegador, sin datos personales, que permite reconstruir el embudo de conversión del *back-office*.

Consumo en la Tarea 2.

- *Lectura incremental:* `GET api/admin/eventos.php?desde_id=N` devuelve los eventos posteriores al último leído; el consumidor guarda el último id y usa el `uuid` para descartar duplicados.
- *Exportación* en CSV o JSON desde el *back-office* para hojas de cálculo o herramientas de BI.

- **Integraciones previstas:** `order.created` y `payment.simulated` → ERP (reserva de stock y facturación); `support.requested` → CRM; `order.status_changed` → avisos al cliente; embudo `product.viewed` → `payment.simulated` → analítica comercial.

4 Limitaciones de la solución

- **Pago y clientes:** el pago es simulado (sin pasarela real, ni en modo pruebas) y no hay cuentas de cliente: el pedido se consulta con código y email, y el carrito solo existe en el navegador.
- **Seguridad:** una única cuenta de *back-office* de prueba, sin roles ni recuperación de contraseña (figura en el README); sin límite de peticiones, por lo que los eventos del navegador podrían falsificarse o enviarse masivamente; la protección CSRF se basa en `SameSite=Strict`, no en un *token*.
- **Operativa:** no se envían correos ni hay integración logística; los estados posteriores al pago se cambian a mano. Las pruebas automáticas solo cubren las reglas de negocio (`tests/pruebas_reglas_negocio.php`); el resto se verificó a mano (Anexo A).
- **Contenido:** fotografías de producto de baja resolución.
- **Hosting provisional:** InfinityFree usa un subdominio genérico y bloquea con JavaScript las peticiones que no vienen de un navegador, por lo que un consumidor externo (Tarea 2) no podría leer la API allí; en DonDominio no existe esa restricción.

5 Uso declarado de herramientas de IA generativa

Se ha usado IA generativa en la versión inicial de la interfaz y, después, Claude Code (Anthropic) como asistente para el análisis del enunciado, el diseño de la arquitectura y del modelo de datos, la programación, los datos de prueba, las pruebas y el borrador de esta memoria y del README. El grupo ha revisado, probado y modificado el resultado. El detalle exigido por el enunciado figura en el Anexo A.

6 Declaración del punto de partida

El proyecto **no parte de ningún repositorio ni plantilla de terceros**. Su punto de partida es una **versión inicial generada mediante IA**: un único HTML (`docs/version_inicial/NOCTILUZ -- Tienda.html`) con portada, catálogo, ficha de producto y un carrito en el navegador, sin servidor ni persistencia y con el filtro de modo sin funcionar. Fue generada por Roberto Ciller con ChatGPT (OpenAI), y Luis Galindo la subió al repositorio. De ella se reutilizó el diseño visual y, con Claude Code, se desarrolló el resto (servidor PHP, modelo de datos, *checkout*, pago simulado, pedidos y estados, soporte, *back-office* y eventos). El primer *commit* del repositorio contiene la versión inicial sin modificar, para que la evolución posterior sea trazable.

A Uso de IA generativa

A.1 Herramientas y tareas

- ChatGPT (OpenAI), usado por Roberto Ciller para generar el HTML inicial; y Claude Code (Anthropic) durante el desarrollo y el despliegue (septiembre y octubre de 2026).
- **Tareas:** revisión del enunciado y de requisitos frente a la versión inicial; propuesta de arquitectura, modelo de datos, reglas de negocio y eventos; programación del servidor, ampliación de la interfaz y datos de prueba; pruebas funcionales en local y borradores del README y de esta memoria.

A.2 Partes relevantes asistidas por IA

Parte	Archivos
Interfaz inicial (diseño, catálogo, carrito)	<code>docs/version_inicial/NOCTILUZ -- Tienda.html</code>
Lógica de negocio, persistencia y API	<code>public_html/app/*.php</code> , <code>public_html/api/*.php</code> , <code>api/admin/*.php</code>
Interfaz ampliada y <i>back-office</i>	<code>index.html</code> , <code>admin.html</code> , <code>assets/js/*.js</code> , <code>assets/css/*.css</code>
Base de datos y datos de prueba	<code>database/*.sql</code> , <code>database/instalar_local.php</code>
Documentación	<code>README.md</code> , borrador de esta memoria, diagramas de las Figuras 1 y 2

A.3 Errores detectados en las respuestas de la IA

- La versión inicial no cumplía el enunciado: sin persistencia ni pedido (el botón de compra solo mostraba un aviso) y con un filtro por modo que no filtraba, al asumir que todas las prendas admitían los tres modos.
- En la ficha de producto, los manejadores de eventos se acumulaban al cambiar de color y un clic se procesaba dos veces; se corrigió anulándolos en cada cambio de vista.
- Animar las 16 ilustraciones a la vez obligaba a repintar continuamente; se limitó a la imagen principal y a la tarjeta señalada.
- Un estilo anulaba el margen lateral de las páginas interiores y el historial del pedido mostraba un texto duplicado.
- Entorno: una ruta mal escapada en la configuración de PHP y un problema de tildes al generar el diagrama.
- Despliegue: el primer ZIP guardó rutas con barra invertida (`app\Db.php`) y el servidor Linux solo extrajo los archivos de la raíz; y el PHP del hosting devolvía importes como 14.9000000000000003552, corregido fijando `serialize_precision` en el arranque de la API.
- La IA propuso Vercel como opción provisional; se descartó al comprobar que no ejecuta PHP nativo ni ofrece MySQL.
- Errores que el grupo detectó al revisar y probar el resultado:
 - Un producto admitía varios modos de iluminación a la vez; se dejó un solo modo por producto.
 - Desde el estado `pagado` no se podía marcar un pedido como enviado o entregado; se añadió esa transición.
 - La etiqueta de modo de las tarjetas del catálogo se quitó y hubo que volver a mostrarla.
 - El navegador mostraba versiones antiguas de la web por la caché; se evitó forzando la recarga.
 - La portada no se adaptaba al móvil ni a la tablet.
 - Las fotos de producto eran provisionales o ilustraciones; se sustituyeron por fotografías.

A.4 Cambios realizados por el grupo

- **Luis Galindo:** despliegue en el hosting (cuenta, base MySQL, importación de esquema y datos, subida de archivos) y pruebas de compra y *back-office* en la URL pública.
- **Javier Zamora:** pruebas automáticas de las reglas de negocio y su documentación, renombrado de productos, un solo modo de iluminación por producto, y fotos y rediseño de las chaquetas.
- **Jun-Jie Jiang Yao:** fondo de la web, foto de la camiseta cian, paso a enviado o entregado desde `pagado` y atajos del resumen del *back-office* a los pedidos de cada estado.
- **Roberto Ciller:** HTML inicial generado con ChatGPT, fotos y descripciones de las mochilas y de la sección «Hecha para la noche», pie de página y acciones en la web.

A.5 Validación del resultado final

- Pruebas automáticas de las reglas de negocio (totales, IVA, envío, cupones y stock) con `tests/pruebas_reglas_negocio.php`.
- Pruebas manuales del flujo completo en local: filtros, ficha, carrito, *checkout*, cupones, envío exprés, pago rechazado y aprobado, seguimiento, soporte y *back-office*; cancelar un pedido devuelve el stock y la vista móvil no se desborda.
- Importes comprobados a mano: 247,00 € con `NOCHE10` → descuento 24,70 €, envío gratis, total 222,30 €, base 183,72 € e IVA 38,58 €.
- Peticiones directas a la API saltándose la interfaz: datos inválidos, modos no admitidos, productos agotados, precios manipulados (se ignoran), transiciones prohibidas y acceso al *back-office* sin sesión.
- En el hosting (noctiluz.free.je): el catálogo carga los 16 productos desde MySQL; `app/config.php`, `app/Db.php` y `datos/` responden 403; la API del *back-office* responde 401 sin sesión; 149 € con `NOCHE10` da 134,10 € (base 110,83 €, IVA 23,27 €); y se completó una compra con pago aprobado y su revisión en el *back-office*, con pedido y eventos.